

```
1 # coding=utf-8
2 # Copyright 2022 EleutherAI and the HuggingFace Inc. team. All rights
  reserved.
3 #
4 # This code is based on EleutherAI's GPT-NeoX library and the GPT-NeoX
5 # and OPT implementations in this library. It has been modified from its
6 # original forms to accommodate minor architectural differences compared
7 # to GPT-NeoX and OPT used by the Meta AI team that trained the model.
8 #
9 # Licensed under the Apache License, Version 2.0 (the "License");
10 # you may not use this file except in compliance with the License.
11 # You may obtain a copy of the License at
12 #
13 #     http://www.apache.org/licenses/LICENSE-2.0
14 #
15 # Unless required by applicable law or agreed to in writing, software
16 # distributed under the License is distributed on an "AS IS" BASIS,
17 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
18 # See the License for the specific language governing permissions and
19 # limitations under the License.
20 from typing import Callable, List, Optional, Tuple, Union
21
22 import torch
23 import torch.utils.checkpoint
24 from torch import nn
25
26 from ...activations import ACT2FN
27 from ...cache_utils import Cache, DynamicCache, StaticCache
28 from ...generation import GenerationMixin
29 from ...modeling_attn_mask_utils import AttentionMaskConverter
30 from ...modeling_flash_attention_utils import FlashAttentionKwargs
31 from ...modeling_outputs import (
32     BaseModelOutputWithPast,
33     CausalLMOutputWithPast,
34     QuestionAnsweringModelOutput,
35     SequenceClassifierOutputWithPast,
36     TokenClassifierOutput,
37 )
38 from ...modeling_rope_utils import ROPE_INIT_FUNCTIONS
39 from ...modeling_utils import ALL_ATTENTION_FUNCTIONS, PreTrainedModel
40 from ...processing_utils import Unpack
41 from ...pytorch_utils import ALL_LAYERNORM_LAYERS
42 from ...utils import (
43     LossKwargs,
44     add_code_sample_docstrings,
45     add_start_docstrings,
46     add_start_docstrings_to_model_forward,
47     logging,
48     replace_return_docstrings,
49 )
50 from ...utils.deprecation import deprecate_kwarg
51 from .configuration_llama import LlamaConfig
52
53
54 logger = logging.get_logger(__name__)
55
56 _CHECKPOINT_FOR_DOC = "meta-llama/Llama-2-7b-hf"
57 _CONFIG_FOR_DOC = "LlamaConfig"
58
59
```

```

60 class LlamaRMSNorm(nn.Module):
61     def __init__(self, hidden_size, eps=1e-6):
62         """
63         LlamaRMSNorm is equivalent to T5LayerNorm
64         """
65         super().__init__()
66         self.weight = nn.Parameter(torch.ones(hidden_size))
67         self.variance_epsilon = eps
68
69     def forward(self, hidden_states):
70         input_dtype = hidden_states.dtype
71         hidden_states = hidden_states.to(torch.float32)
72         variance = hidden_states.pow(2).mean(-1, keepdim=True)
73         hidden_states = hidden_states * torch.rsqrt(variance +
self.variance_epsilon)
74         return self.weight * hidden_states.to(input_dtype)
75
76     def extra_repr(self):
77         return f"{tuple(self.weight.shape)}, eps={self.variance_epsilon}"
78
79
80 ALL_LAYERNORM_LAYERS.append(LlamaRMSNorm)
81
82
83 class LlamaRotaryEmbedding(nn.Module):
84     def __init__(self, config: LlamaConfig, device=None):
85         super().__init__()
86         # BC: "rope_type" was originally "type"
87         if hasattr(config, "rope_scaling") and config.rope_scaling is not
None:
88             self.rope_type = config.rope_scaling.get("rope_type",
config.rope_scaling.get("type"))
89         else:
90             self.rope_type = "default"
91         self.max_seq_len_cached = config.max_position_embeddings
92         self.original_max_seq_len = config.max_position_embeddings
93
94         self.config = config
95         self.rope_init_fn = ROPE_INIT_FUNCTIONS[self.rope_type]
96
97         inv_freq, self.attention_scaling = self.rope_init_fn(self.config,
device)
98         self.register_buffer("inv_freq", inv_freq, persistent=False)
99         self.original_inv_freq = self.inv_freq
100
101     def _dynamic_frequency_update(self, position_ids, device):
102         """
103         dynamic RoPE layers should recompute `inv_freq` in the following
situations:
104         1 - growing beyond the cached sequence length (allow scaling)
105         2 - the current sequence length is in the original scale (avoid
losing precision with small sequences)
106         """
107         seq_len = torch.max(position_ids) + 1
108         if seq_len > self.max_seq_len_cached: # growth
109             inv_freq, self.attention_scaling =
self.rope_init_fn(self.config, device, seq_len=seq_len)
110             self.register_buffer("inv_freq", inv_freq, persistent=False) #
TODO joao: may break with compilation
111             self.max_seq_len_cached = seq_len

```

```

112
113         if seq_len < self.original_max_seq_len and self.max_seq_len_cached >
self.original_max_seq_len: # reset
114             # This .to() is needed if the model has been moved to a device
after being initialized (because
115             # the buffer is automatically moved, but not the original copy)
116             self.original_inv_freq = self.original_inv_freq.to(device)
117             self.register_buffer("inv_freq", self.original_inv_freq,
persistent=False)
118             self.max_seq_len_cached = self.original_max_seq_len
119
120     @torch.no_grad()
121     def forward(self, x, position_ids):
122         if "dynamic" in self.rope_type:
123             self._dynamic_frequency_update(position_ids, device=x.device)
124
125         # Core RoPE block
126         inv_freq_expanded = self.inv_freq[None, :,
None].float().expand(position_ids.shape[0], -1, 1)
127         position_ids_expanded = position_ids[:, None, :].float()
128         # Force float32 (see
https://github.com/huggingface/transformers/pull/29285)
129         device_type = x.device.type
130         device_type = device_type if isinstance(device_type, str) and
device_type != "mps" else "cpu"
131         with torch.autocast(device_type=device_type, enabled=False):
132             freqs = (inv_freq_expanded.float() @
position_ids_expanded.float()).transpose(1, 2)
133             emb = torch.cat((freqs, freqs), dim=-1)
134             cos = emb.cos()
135             sin = emb.sin()
136
137         # Advanced RoPE types (e.g. yarn) apply a post-processing scaling
factor, equivalent to scaling attention
138         cos = cos * self.attention_scaling
139         sin = sin * self.attention_scaling
140
141         return cos.to(dtype=x.dtype), sin.to(dtype=x.dtype)
142
143
144 def rotate_half(x):
145     """Rotates half the hidden dims of the input."""
146     x1 = x[..., : x.shape[-1] // 2]
147     x2 = x[..., x.shape[-1] // 2 :]
148     return torch.cat((-x2, x1), dim=-1)
149
150
151 def apply_rotary_pos_emb(q, k, cos, sin, position_ids=None,
unsqueeze_dim=1):
152     """Applies Rotary Position Embedding to the query and key tensors.
153
154     Args:
155         q (`torch.Tensor`): The query tensor.
156         k (`torch.Tensor`): The key tensor.
157         cos (`torch.Tensor`): The cosine part of the rotary embedding.
158         sin (`torch.Tensor`): The sine part of the rotary embedding.
159         position_ids (`torch.Tensor`, *optional*):
            Deprecated and unused.
160         unsqueeze_dim (`int`, *optional*, defaults to 1):

```

```

162         The 'unsqueeze_dim' argument specifies the dimension along which
to unsqueeze cos[position_ids] and
163         sin[position_ids] so that they can be properly broadcasted to
the dimensions of q and k. For example, note
164         that cos[position_ids] and sin[position_ids] have the shape
[batch_size, seq_len, head_dim]. Then, if q and
165         k have the shape [batch_size, heads, seq_len, head_dim], then
setting unsqueeze_dim=1 makes
166         cos[position_ids] and sin[position_ids] broadcastable to the
shapes of q and k. Similarly, if q and k have
167         the shape [batch_size, seq_len, heads, head_dim], then set
unsqueeze_dim=2.
168     Returns:
169         `tuple(torch.Tensor)` comprising of the query and key tensors
rotated using the Rotary Position Embedding.
170     """
171     cos = cos.unsqueeze(unsqueeze_dim)
172     sin = sin.unsqueeze(unsqueeze_dim)
173     q_embed = (q * cos) + (rotate_half(q) * sin)
174     k_embed = (k * cos) + (rotate_half(k) * sin)
175     return q_embed, k_embed
176
177
178 class LlamaMLP(nn.Module):
179     def __init__(self, config):
180         super().__init__()
181         self.config = config
182         self.hidden_size = config.hidden_size
183         self.intermediate_size = config.intermediate_size
184         self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size,
bias=config.mlp_bias)
185         self.up_proj = nn.Linear(self.hidden_size, self.intermediate_size,
bias=config.mlp_bias)
186         self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size,
bias=config.mlp_bias)
187         self.act_fn = ACT2FN[config.hidden_act]
188
189     def forward(self, x):
190         down_proj = self.down_proj(self.act_fn(self.gate_proj(x)) *
self.up_proj(x))
191         return down_proj
192
193
194 def repeat_kv(hidden_states: torch.Tensor, n_rep: int) -> torch.Tensor:
195     """
196     This is the equivalent of torch.repeat_interleave(x, dim=1,
repeats=n_rep). The hidden states go from (batch,
197     num_key_value_heads, seq_len, head_dim) to (batch, num_attention_heads,
seq_len, head_dim)
198     """
199     batch, num_key_value_heads, slen, head_dim = hidden_states.shape
200     if n_rep == 1:
201         return hidden_states
202     hidden_states = hidden_states[:, :, None, :, :].expand(batch,
num_key_value_heads, n_rep, slen, head_dim)
203     return hidden_states.reshape(batch, num_key_value_heads * n_rep, slen,
head_dim)
204
205
206 def eager_attention_forward(

```

```

207     module: nn.Module,
208     query: torch.Tensor,
209     key: torch.Tensor,
210     value: torch.Tensor,
211     attention_mask: Optional[torch.Tensor],
212     scaling: float,
213     dropout: float = 0.0,
214     **kwargs,
215 ):
216     key_states = repeat_kv(key, module.num_key_value_groups)
217     value_states = repeat_kv(value, module.num_key_value_groups)
218
219     attn_weights = torch.matmul(query, key_states.transpose(2, 3)) * scaling
220     if attention_mask is not None:
221         causal_mask = attention_mask[:, :, :, : key_states.shape[-2]]
222         attn_weights = attn_weights + causal_mask
223
224     attn_weights = nn.functional.softmax(attn_weights, dim=-1,
dtype=torch.float32).to(query.dtype)
225     attn_weights = nn.functional.dropout(attn_weights, p=dropout,
training=module.training)
226     attn_output = torch.matmul(attn_weights, value_states)
227     attn_output = attn_output.transpose(1, 2).contiguous()
228
229     return attn_output, attn_weights
230
231
232 class LlamaAttention(nn.Module):
233     """Multi-headed attention from 'Attention Is All You Need' paper"""
234
235     def __init__(self, config: LlamaConfig, layer_idx: int):
236         super().__init__()
237         self.config = config
238         self.layer_idx = layer_idx
239         # 4096 / 32 = 128
240         self.head_dim = getattr(config, "head_dim", config.hidden_size //
config.num_attention_heads)
241         # 1
242         self.num_key_value_groups = config.num_attention_heads //
config.num_key_value_heads
243         self.scaling = self.head_dim**-0.5
244         self.attention_dropout = config.attention_dropout
245         self.is_causal = True
246
247
248         # vocab_size=32000,
249         # hidden_size=4096,
250         # intermediate_size=11008,
251         # num_hidden_layers=32,
252         # num_attention_heads=32,
253         # num_key_value_heads=None,
254
255         self.q_proj = nn.Linear(
256             config.hidden_size, config.num_attention_heads * self.head_dim,
bias=config.attention_bias
257         )
258         self.k_proj = nn.Linear(
259             config.hidden_size, config.num_key_value_heads * self.head_dim,
bias=config.attention_bias
260         )

```

```

261         self.v_proj = nn.Linear(
262             config.hidden_size, config.num_key_value_heads * self.head_dim,
bias=config.attention_bias
263         )
264         self.o_proj = nn.Linear(
265             config.num_attention_heads * self.head_dim, config.hidden_size,
bias=config.attention_bias
266         )
267
268     def forward(
269         self,
270         hidden_states: torch.Tensor,
271         position_embeddings: Tuple[torch.Tensor, torch.Tensor],
272         attention_mask: Optional[torch.Tensor],
273         past_key_value: Optional[Cache] = None,
274         cache_position: Optional[torch.LongTensor] = None,
275         **kwargs: Unpack[FlashAttentionKwargs],
276     ) -> Tuple[torch.Tensor, Optional[torch.Tensor],
Optional[Tuple[torch.Tensor]]]:
277         input_shape = hidden_states.shape[:-1]
278         hidden_shape = (*input_shape, -1, self.head_dim)
279
280         query_states =
self.q_proj(hidden_states).view(hidden_shape).transpose(1, 2)
281         key_states =
self.k_proj(hidden_states).view(hidden_shape).transpose(1, 2)
282         value_states =
self.v_proj(hidden_states).view(hidden_shape).transpose(1, 2)
283
284         cos, sin = position_embeddings
285         query_states, key_states = apply_rotary_pos_emb(query_states,
key_states, cos, sin)
286
287         if past_key_value is not None:
288             # sin and cos are specific to RoPE models; cache_position needed
for the static cache
289             cache_kwargs = {"sin": sin, "cos": cos, "cache_position":
cache_position}
290             key_states, value_states = past_key_value.update(key_states,
value_states, self.layer_idx, cache_kwargs)
291
292         attention_interface: Callable = eager_attention_forward
293         if self.config._attn_implementation != "eager":
294             if self.config._attn_implementation == "sdpa" and
kwargs.get("output_attentions", False):
295                 logger.warning_once(
296                     "`torch.nn.functional.scaled_dot_product_attention` does
not support `output_attentions=True`. Falling back to "
297                     'eager attention. This warning can be removed using the
argument `attn_implementation="eager"` when loading the model.'
298                 )
299             else:
300                 attention_interface =
ALL_ATTENTION_FUNCTIONS[self.config._attn_implementation]
301
302         attn_output, attn_weights = attention_interface(
303             self,
304             query_states,
305             key_states,
306             value_states,

```



```

307         attention_mask,
308         dropout=0.0 if not self.training else self.attention_dropout,
309         scaling=self.scaling,
310         **kwargs,
311     )
312
313     attn_output = attn_output.reshape(*input_shape, -1).contiguous()
314     attn_output = self.o_proj(attn_output)
315     return attn_output, attn_weights
316
317
318 class LlamaDecoderLayer(nn.Module):
319     def __init__(self, config: LlamaConfig, layer_idx: int):
320         super().__init__()
321         self.hidden_size = config.hidden_size
322
323         self.self_attn = LlamaAttention(config=config, layer_idx=layer_idx)
324         self.mlp = LlamaMLP(config)
325         self.input_layernorm = LlamaRMSNorm(config.hidden_size,
eps=config.rms_norm_eps)
326         self.post_attention_layernorm = LlamaRMSNorm(config.hidden_size,
eps=config.rms_norm_eps)
327
328     def forward(
329         self,
330         hidden_states: torch.Tensor,
331         attention_mask: Optional[torch.Tensor] = None,
332         position_ids: Optional[torch.LongTensor] = None,
333         past_key_value: Optional[Cache] = None,
334         output_attentions: Optional[bool] = False,
335         use_cache: Optional[bool] = False,
336         cache_position: Optional[torch.LongTensor] = None,
337         position_embeddings: Optional[Tuple[torch.Tensor, torch.Tensor]] =
None, # necessary, but kept here for BC
338         **kwargs: Unpack[FlashAttentionKwargs],
339     ) -> Tuple[torch.FloatTensor, Optional[Tuple[torch.FloatTensor,
torch.FloatTensor]]]:
340         residual = hidden_states
341
342         hidden_states = self.input_layernorm(hidden_states)
343
344         # Self Attention
345         hidden_states, self_attn_weights = self.self_attn(
346             hidden_states=hidden_states,
347             attention_mask=attention_mask,
348             position_ids=position_ids,
349             past_key_value=past_key_value,
350             output_attentions=output_attentions,
351             use_cache=use_cache,
352             cache_position=cache_position,
353             position_embeddings=position_embeddings,
354             **kwargs,
355         )
356         hidden_states = residual + hidden_states
357
358         # Fully Connected
359         residual = hidden_states
360         hidden_states = self.post_attention_layernorm(hidden_states)
361         hidden_states = self.mlp(hidden_states)
362         hidden_states = residual + hidden_states

```

```

363         outputs = (hidden_states,)
364         if output_attentions:
365             outputs += (self_attn_weights,)
366
367         return outputs
368
369
370
371 LLAMA_START_DOCSTRING = r"""
372     This model inherits from [PreTrainedModel]. Check the superclass
373     documentation for the generic methods the
374     library implements for all its model (such as downloading or saving,
375     resizing the input embeddings, pruning heads
376     etc.)
377
378     This model is also a PyTorch [torch.nn.Module]
379     (https://pytorch.org/docs/stable/nn.html#torch.nn.Module) subclass.
380     Use it as a regular PyTorch Module and refer to the PyTorch
381     documentation for all matter related to general usage
382     and behavior.
383
384     Parameters:
385         config ([LlamaConfig]):
386             Model configuration class with all the parameters of the model.
387             Initializing with a config file does not
388             load the weights associated with the model, only the
389             configuration. Check out the
390             [~PreTrainedModel.from_pretrained] method to load the model
391             weights.
392 """
393
394 @add_start_docstrings(
395     "The bare LLaMA Model outputting raw hidden-states without any specific
396     head on top.",
397     LLAMA_START_DOCSTRING,
398 )
399 class LlamaPreTrainedModel(PreTrainedModel):
400     config_class = LlamaConfig
401     base_model_prefix = "model"
402     supports_gradient_checkpointing = True
403     _no_split_modules = ["LlamaDecoderLayer"]
404     _skip_keys_device_placement = ["past_key_values"]
405     _supports_flash_attn_2 = True
406     _supports_sdpa = True
407     _supports_flex_attn = True
408     _supports_cache_class = True
409     _supports_quantized_cache = True
410     _supports_static_cache = True
411     _supports_attention_backend = True
412
413     def _init_weights(self, module):
414         std = self.config.initializer_range
415         if isinstance(module, nn.Linear):
416             module.weight.data.normal_(mean=0.0, std=std)
417             if module.bias is not None:
418                 module.bias.data.zero_()
419         elif isinstance(module, nn.Embedding):
420             module.weight.data.normal_(mean=0.0, std=std)
421             if module.padding_idx is not None:

```



```

415         module.weight.data[module.padding_idx].zero_()
416
417
418 LLAMA_INPUTS_DOCSTRING = r"""
419     Args:
420         input_ids (`torch.LongTensor` of shape `(batch_size,
sequence_length)`):
421             Indices of input sequence tokens in the vocabulary. Padding will
be ignored by default should you provide
422             it.
423
424             Indices can be obtained using [`AutoTokenizer`]. See
[`PreTrainedTokenizer.encode`] and
425             [`PreTrainedTokenizer.__call__`] for details.
426
427             [What are input IDs?](../glossary#input-ids)
428         attention_mask (`torch.Tensor` of shape `(batch_size,
sequence_length)`, *optional*):
429             Mask to avoid performing attention on padding token indices.
Mask values selected in `[0, 1]`:
430
431             - 1 for tokens that are **not masked**,
432             - 0 for tokens that are **masked**.
433
434             [What are attention masks?](../glossary#attention-mask)
435
436             Indices can be obtained using [`AutoTokenizer`]. See
[`PreTrainedTokenizer.encode`] and
437             [`PreTrainedTokenizer.__call__`] for details.
438
439             If `past_key_values` is used, optionally only the last
`input_ids` have to be input (see
440             `past_key_values`).
441
442             If you want to change padding behavior, you should read
[`modeling_opt._prepare_decoder_attention_mask`]
443             and modify to your needs. See diagram 1 in [the paper]
(https://arxiv.org/abs/1910.13461) for more
444             information on the default strategy.
445
446             - 1 indicates the head is **not masked**,
447             - 0 indicates the head is **masked**.
448         position_ids (`torch.LongTensor` of shape `(batch_size,
sequence_length)`, *optional*):
449             Indices of positions of each input sequence tokens in the
position embeddings. Selected in the range `[0,
450             config.n_positions - 1]`.
451
452             [What are position IDs?](../glossary#position-ids)
453         past_key_values (`Cache` or `tuple(tuple(torch.FloatTensor))`,
*optional*):
454             Pre-computed hidden-states (key and values in the self-attention
blocks and in the cross-attention
455             blocks) that can be used to speed up sequential decoding. This
typically consists in the `past_key_values`
456             returned by the model at a previous stage of decoding, when
`use_cache=True` or `config.use_cache=True`.
457
458             Two formats are allowed:
459             - a [`~cache_utils.Cache`] instance, see our

```

```

460         [kv cache guide]
461         (https://huggingface.co/docs/transformers/en/kv\_cache);
462         - Tuple of `tuple(torch.FloatTensor)` of length
463         `config.n_layers`, with each tuple having 2 tensors of
464         shape `(batch_size, num_heads, sequence_length,
465         embed_size_per_head)`. This is also known as the legacy
466         cache format.
467
468         The model will output the same cache format that is fed as
469         input. If no `past_key_values` are passed, the
470         legacy cache format will be returned.
471
472         If `past_key_values` are used, the user can optionally input
473         only the last `input_ids` (those that don't
474         have their past key value states given to this model) of shape
475         `(batch_size, 1)` instead of all `input_ids`
476         of shape `(batch_size, sequence_length)`.
477         inputs_embeds (`torch.FloatTensor` of shape `(batch_size,
478         sequence_length, hidden_size)`, *optional*):
479         Optionally, instead of passing `input_ids` you can choose to
480         directly pass an embedded representation. This
481         is useful if you want more control over how to convert
482         `input_ids` indices into associated vectors than the
483         model's internal embedding lookup matrix.
484         use_cache (`bool`, *optional*):
485         If set to `True`, `past_key_values` key value states are
486         returned and can be used to speed up decoding (see
487         `past_key_values`).
488         output_attentions (`bool`, *optional*):
489         Whether or not to return the attentions tensors of all attention
490         layers. See `attentions` under returned
491         tensors for more detail.
492         output_hidden_states (`bool`, *optional*):
493         Whether or not to return the hidden states of all layers. See
494         `hidden_states` under returned tensors for
495         more detail.
496         return_dict (`bool`, *optional*):
497         Whether or not to return a [~utils.ModelOutput] instead of a
498         plain tuple.
499         cache_position (`torch.LongTensor` of shape `(sequence_length)`,
500         *optional*):
501         Indices depicting the position of the input sequence tokens in
502         the sequence. Contrarily to `position_ids`,
503         this tensor is not affected by padding. It is used to update the
504         cache in the correct position and to infer
505         the complete sequence length.
506     """
507
508     @add_start_docstrings(
509         "The bare LLaMA Model outputting raw hidden-states without any specific
510         head on top.",
511         LLAMA_START_DOCSTRING,
512     )
513     class LlamaModel(LlamaPreTrainedModel):
514         """
515         Transformer decoder consisting of *config.num_hidden_layers* layers.
516         Each layer is a [`LlamaDecoderLayer`]
517
518         Args:

```

```

502         config: LlamaConfig
503     """
504
505     def __init__(self, config: LlamaConfig):
506         super().__init__(config)
507         self.padding_idx = config.pad_token_id
508         self.vocab_size = config.vocab_size
509
510         self.embed_tokens = nn.Embedding(config.vocab_size,
config.hidden_size, self.padding_idx)
511         self.layers = nn.ModuleList(
512             [LlamaDecoderLayer(config, layer_idx) for layer_idx in
range(config.num_hidden_layers)]
513         )
514         self.norm = LlamaRMSNorm(config.hidden_size,
eps=config.rms_norm_eps)
515         self.rotary_emb = LlamaRotaryEmbedding(config=config)
516         self.gradient_checkpointing = False
517
518         # Initialize weights and apply final processing
519         self.post_init()
520
521     def get_input_embeddings(self):
522         return self.embed_tokens
523
524     def set_input_embeddings(self, value):
525         self.embed_tokens = value
526
527     @add_start_docstrings_to_model_forward(LLAMA_INPUTS_DOCSTRING)
528     def forward(
529         self,
530         input_ids: torch.LongTensor = None,
531         attention_mask: Optional[torch.Tensor] = None,
532         position_ids: Optional[torch.LongTensor] = None,
533         past_key_values: Optional[Cache] = None,
534         inputs_embeds: Optional[torch.FloatTensor] = None,
535         use_cache: Optional[bool] = None,
536         output_attentions: Optional[bool] = None,
537         output_hidden_states: Optional[bool] = None,
538         return_dict: Optional[bool] = None,
539         cache_position: Optional[torch.LongTensor] = None,
540         **flash_attn_kwargs: Unpack[FlashAttentionKwargs],
541     ) -> Union[Tuple, BaseModelOutputWithPast]:
542         output_attentions = output_attentions if output_attentions is not
None else self.config.output_attentions
543         output_hidden_states = (
544             output_hidden_states if output_hidden_states is not None else
self.config.output_hidden_states
545         )
546         use_cache = use_cache if use_cache is not None else
self.config.use_cache
547         return_dict = return_dict if return_dict is not None else
self.config.use_return_dict
548
549         if (input_ids is None) ^ (inputs_embeds is not None):
550             raise ValueError("You must specify exactly one of input_ids or
inputs_embeds")
551
552         if self.gradient_checkpointing and self.training and use_cache:
553

```

```

554         "`use_cache=True` is incompatible with gradient
checkpointing. Setting `use_cache=False`."
555     )
556     use_cache = False
557
558     if inputs_embeds is None:
559         inputs_embeds = self.embed_tokens(input_ids)
560
561     if use_cache and past_key_values is None:
562         past_key_values = DynamicCache()
563
564     if cache_position is None:
565         past_seen_tokens = past_key_values.get_seq_length() if
past_key_values is not None else 0
566         cache_position = torch.arange(
567             past_seen_tokens, past_seen_tokens + inputs_embeds.shape[1],
device=inputs_embeds.device
568         )
569
570     if position_ids is None:
571         position_ids = cache_position.unsqueeze(0)
572
573     causal_mask = self._update_causal_mask(
574         attention_mask, inputs_embeds, cache_position, past_key_values,
output_attentions
575     )
576
577     hidden_states = inputs_embeds
578
579     # create position embeddings to be shared across the decoder layers
580     position_embeddings = self.rotary_emb(hidden_states, position_ids)
581
582     # decoder layers
583     all_hidden_states = () if output_hidden_states else None
584     all_self_attns = () if output_attentions else None
585
586     for decoder_layer in self.layers[: self.config.num_hidden_layers]:
587         if output_hidden_states:
588             all_hidden_states += (hidden_states,)
589
590         if self.gradient_checkpointing and self.training:
591             layer_outputs = self._gradient_checkpointing_func(
592                 decoder_layer.__call__,
593                 hidden_states,
594                 causal_mask,
595                 position_ids,
596                 past_key_values,
597                 output_attentions,
598                 use_cache,
599                 cache_position,
600                 position_embeddings,
601             )
602         else:
603             layer_outputs = decoder_layer(
604                 hidden_states,
605                 attention_mask=causal_mask,
606                 position_ids=position_ids,
607                 past_key_value=past_key_values,
608                 output_attentions=output_attentions,
609                 use_cache=use_cache,

```

```

610         cache_position=cache_position,
611         position_embeddings=position_embeddings,
612         **flash_attn_kwargs,
613     )
614
615     hidden_states = layer_outputs[0]
616
617     if output_attentions:
618         all_self_attns += (layer_outputs[1],)
619
620     hidden_states = self.norm(hidden_states)
621
622     # add hidden states from the last decoder layer
623     if output_hidden_states:
624         all_hidden_states += (hidden_states,)
625
626     output = BaseModelOutputWithPast(
627         last_hidden_state=hidden_states,
628         past_key_values=past_key_values if use_cache else None,
629         hidden_states=all_hidden_states,
630         attentions=all_self_attns,
631     )
632     return output if return_dict else output.to_tuple()
633
634     def _update_causal_mask(
635         self,
636         attention_mask: torch.Tensor,
637         input_tensor: torch.Tensor,
638         cache_position: torch.Tensor,
639         past_key_values: Cache,
640         output_attentions: bool,
641     ):
642         if self.config._attn_implementation == "flash_attention_2":
643             if attention_mask is not None and (attention_mask == 0.0).any():
644                 return attention_mask
645             return None
646
647         # For SDPA, when possible, we will rely on its `is_causal` argument
648         # instead of its `attn_mask` argument, in
649         # order to dispatch on Flash Attention 2. This feature is not
650         # compatible with static cache, as SDPA will fail
651         # to infer the attention mask.
652         past_seen_tokens = past_key_values.get_seq_length() if
653         past_key_values is not None else 0
654         using_static_cache = isinstance(past_key_values, StaticCache)
655
656         # When output attentions is True, sdpa implementation's forward
657         # method calls the eager implementation's forward
658         if self.config._attn_implementation == "sdpa" and not
659         using_static_cache and not output_attentions:
660             if AttentionMaskConverter._ignore_causal_mask_sdpa(
661                 attention_mask,
662                 inputs_embeds=input_tensor,
663                 past_key_values_length=past_seen_tokens,
664                 is_training=self.training,
665             ):
666                 return None
667
668         dtype, device = input_tensor.dtype, input_tensor.device
669         sequence_length = input_tensor.shape[1]

```

```

665         if using_static_cache:
666             target_length = past_key_values.get_max_cache_shape()
667         else:
668             target_length = (
669                 attention_mask.shape[-1]
670                 if isinstance(attention_mask, torch.Tensor)
671                 else past_seen_tokens + sequence_length + 1
672             )
673
674         # In case the provided `attention` mask is 2D, we generate a causal
mask here (4D).
675         causal_mask =
self._prepare_4d_causal_attention_mask_with_cache_position(
676             attention_mask,
677             sequence_length=sequence_length,
678             target_length=target_length,
679             dtype=dtype,
680             device=device,
681             cache_position=cache_position,
682             batch_size=input_tensor.shape[0],
683         )
684
685         if (
686             self.config.attn_implementation == "sdpa"
687             and attention_mask is not None
688             and attention_mask.device.type in ["cuda", "xpu"]
689             and not output_attentions
690         ):
691             # Attend to all tokens in fully masked rows in the causal_mask,
for example the relevant first rows when
692             # using left padding. This is required by
F.scaled_dot_product_attention memory-efficient attention path.
693             # Details: https://github.com/pytorch/pytorch/issues/110213
694             min_dtype = torch.finfo(dtype).min
695             causal_mask =
AttentionMaskConverter._unmask_unattended(causal_mask, min_dtype)
696
697         return causal_mask
698
699     @staticmethod
700     def _prepare_4d_causal_attention_mask_with_cache_position(
701         attention_mask: torch.Tensor,
702         sequence_length: int,
703         target_length: int,
704         dtype: torch.dtype,
705         device: torch.device,
706         cache_position: torch.Tensor,
707         batch_size: int,
708         **kwargs,
709     ):
710         """
711         Creates a causal 4D mask of shape `(batch_size, 1, query_length,
key_value_length)` from a 2D mask of shape
712         `(batch_size, key_value_length)`, or if the input `attention_mask`
is already 4D, do nothing.
713
714         Args:
715             attention_mask (`torch.Tensor`):
716                 A 2D attention mask of shape `(batch_size,
key_value_length)` or a 4D attention mask of shape

```



```

717         `(batch_size, 1, query_length, key_value_length)`.
718     sequence_length (`int`):
719         The sequence length being processed.
720     target_length (`int`):
721         The target length: when generating with static cache, the
mask should be as long as the static cache,
722         to account for the 0 padding, the part of the cache that is
not filled yet.
723     dtype (`torch.dtype`):
724         The dtype to use for the 4D attention mask.
725     device (`torch.device`):
726         The device to place the 4D attention mask on.
727     cache_position (`torch.Tensor`):
728         Indices depicting the position of the input sequence tokens
in the sequence.
729     batch_size (`torch.Tensor`):
730         Batch size.
731     """
732     if attention_mask is not None and attention_mask.dim() == 4:
733         # In this case we assume that the mask comes already in inverted
form and requires no inversion or slicing.
734         causal_mask = attention_mask
735     else:
736         min_dtype = torch.finfo(dtype).min
737         causal_mask = torch.full(
738             (sequence_length, target_length), fill_value=min_dtype,
dtype=dtype, device=device
739         )
740         if sequence_length != 1:
741             causal_mask = torch.triu(causal_mask, diagonal=1)
742             causal_mask *= torch.arange(target_length, device=device) >
cache_position.reshape(-1, 1)
743             causal_mask = causal_mask[None, None, :, :].expand(batch_size,
1, -1, -1)
744         if attention_mask is not None:
745             causal_mask = causal_mask.clone() # copy to contiguous
memory for in-place edit
746             mask_length = attention_mask.shape[-1]
747             padding_mask = causal_mask[:, :, :, :mask_length] +
attention_mask[:, None, None, :].to(
748                 causal_mask.device
749             )
750             padding_mask = padding_mask == 0
751             causal_mask[:, :, :, :mask_length] = causal_mask[:, :, :,
:mask_length].masked_fill(
752                 padding_mask, min_dtype
753             )
754
755     return causal_mask
756
757
758 class KwargForCausalLM(FlashAttentionKwargs, LossKwargs): ...
759
760
761 class LlamaForCausalLM(LlamaPreTrainedModel, GenerationMixin):
762     _tied_weights_keys = ["lm_head.weight"]
763     _tp_plan = {"lm_head": "colwise_rep"}
764     _pp_plan = {"lm_head": (["hidden_states"], ["logits"])}
765
766     def __init__(self, config):

```

```

767         super().__init__(config)
768         self.model = LlamaModel(config)
769         self.vocab_size = config.vocab_size
770         self.lm_head = nn.Linear(config.hidden_size, config.vocab_size,
bias=False)
771
772         # Initialize weights and apply final processing
773         self.post_init()
774
775     def get_input_embeddings(self):
776         return self.model.embed_tokens
777
778     def set_input_embeddings(self, value):
779         self.model.embed_tokens = value
780
781     def get_output_embeddings(self):
782         return self.lm_head
783
784     def set_output_embeddings(self, new_embeddings):
785         self.lm_head = new_embeddings
786
787     def set_decoder(self, decoder):
788         self.model = decoder
789
790     def get_decoder(self):
791         return self.model
792
793     @deprecate_kwarg("num_logits_to_keep", version="4.50",
new_name="logits_to_keep")
794     @add_start_docstrings_to_model_forward(LLAMA_INPUTS_DOCSTRING)
795     @replace_return_docstrings(output_type=CausalLMOutputWithPast,
config_class=_CONFIG_FOR_DOC)
796     def forward(
797         self,
798         input_ids: torch.LongTensor = None,
799         attention_mask: Optional[torch.Tensor] = None,
800         position_ids: Optional[torch.LongTensor] = None,
801         past_key_values: Optional[Union[Cache, List[torch.FloatTensor]]] =
None,
802         inputs_embeds: Optional[torch.FloatTensor] = None,
803         labels: Optional[torch.LongTensor] = None,
804         use_cache: Optional[bool] = None,
805         output_attentions: Optional[bool] = None,
806         output_hidden_states: Optional[bool] = None,
807         return_dict: Optional[bool] = None,
808         cache_position: Optional[torch.LongTensor] = None,
809         logits_to_keep: Union[int, torch.Tensor] = 0,
810         **kwargs: Unpack[KwargsForCausalLM],
811     ) -> Union[Tuple, CausalLMOutputWithPast]:
812         r"""
813         Args:
814             labels (`torch.LongTensor` of shape `(batch_size,
sequence_length)`, *optional*):
815                 Labels for computing the masked language modeling loss.
Indices should either be in `[0, ...,
816                 config.vocab_size]` or -100 (see `input_ids` docstring).
Tokens with indices set to `-100` are ignored
817                 (masked), the loss is only computed for the tokens with
labels in `[0, ..., config.vocab_size]`.
818

```

```

819         logits_to_keep (`int` or `torch.Tensor`, *optional*):
820             If an `int`, compute logits for the last `logits_to_keep`
tokens. If `0`, calculate logits for all
821             `input_ids` (special case). Only last token logits are
needed for generation, and calculating them only for that
822             token can save memory, which becomes pretty significant for
long sequences or large vocabulary size.
823             If a `torch.Tensor`, must be 1D corresponding to the indices
to keep in the sequence length dimension.
824             This is useful when using packed tensor format (single
dimension for batch and sequence length).
825
826         Returns:
827
828         Example:
829
830         ```python
831         >>> from transformers import AutoTokenizer, LlamaForCausalLM
832
833         >>> model = LlamaForCausalLM.from_pretrained("meta-llama/Llama-2-7b-
hf")
834         >>> tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-
7b-hf")
835
836         >>> prompt = "Hey, are you conscious? Can you talk to me?"
837         >>> inputs = tokenizer(prompt, return_tensors="pt")
838
839         >>> # Generate
840         >>> generate_ids = model.generate(inputs.input_ids, max_length=30)
841         >>> tokenizer.batch_decode(generate_ids, skip_special_tokens=True,
clean_up_tokenization_spaces=False)[0]
842         "Hey, are you conscious? Can you talk to me?\nI'm not conscious, but
I can talk to you."
843         ```
844         output_attentions = output_attentions if output_attentions is not
None else self.config.output_attentions
845         output_hidden_states = (
846             output_hidden_states if output_hidden_states is not None else
self.config.output_hidden_states
847         )
848         return_dict = return_dict if return_dict is not None else
self.config.use_return_dict
849
850         # decoder outputs consists of (dec_features, layer_state,
dec_hidden, dec_attn)
851         outputs = self.model(
852             input_ids=input_ids,
853             attention_mask=attention_mask,
854             position_ids=position_ids,
855             past_key_values=past_key_values,
856             inputs_embeds=inputs_embeds,
857             use_cache=use_cache,
858             output_attentions=output_attentions,
859             output_hidden_states=output_hidden_states,
860             return_dict=return_dict,
861             cache_position=cache_position,
862             **kwargs,
863         )
864
865         hidden_states = outputs[0]

```

```

866         # Only compute necessary logits, and do not upcast them to float if
we are not computing the loss
867         slice_indices = slice(-logits_to_keep, None) if
isinstance(logits_to_keep, int) else logits_to_keep
868         logits = self.lm_head(hidden_states[:, slice_indices, :])
869
870         loss = None
871         if labels is not None:
872             loss = self.loss_function(logits=logits, labels=labels,
vocab_size=self.config.vocab_size, **kwargs)
873
874         if not return_dict:
875             output = (logits,) + outputs[1:]
876             return (loss,) + output if loss is not None else output
877
878         return CausalLMOutputWithPast(
879             loss=loss,
880             logits=logits,
881             past_key_values=outputs.past_key_values,
882             hidden_states=outputs.hidden_states,
883             attentions=outputs.attentions,
884         )
885
886
887 @add_start_docstrings(
888     """
889     The LLaMa Model transformer with a sequence classification head on top
(linear layer).
890
891     [`LlamaForSequenceClassification`] uses the last token in order to do
the classification, as other causal models
892     (e.g. GPT-2) do.
893
894     Since it does classification on the last token, it requires to know the
position of the last token. If a
895     `pad_token_id` is defined in the configuration, it finds the last token
that is not a padding token in each row. If
896     no `pad_token_id` is defined, it simply takes the last value in each row
of the batch. Since it cannot guess the
897     padding tokens when `inputs_embeds` are passed instead of `input_ids`,
it does the same (take the last value in
898     each row of the batch).
899     """,
900     LLAMA_START_DOCSTRING,
901 )
902 class LlamaForSequenceClassification(LlamaPreTrainedModel):
903     def __init__(self, config):
904         super().__init__(config)
905         self.num_labels = config.num_labels
906         self.model = LlamaModel(config)
907         self.score = nn.Linear(config.hidden_size, self.num_labels,
bias=False)
908
909         # Initialize weights and apply final processing
910         self.post_init()
911
912     def get_input_embeddings(self):
913         return self.model.embed_tokens
914
915     def set_input_embeddings(self, value):

```

```

916         self.model.embed_tokens = value
917
918     @add_start_docstrings_to_model_forward(LLAMA_INPUTS_DOCSTRING)
919     def forward(
920         self,
921         input_ids: Optional[torch.LongTensor] = None,
922         attention_mask: Optional[torch.Tensor] = None,
923         position_ids: Optional[torch.LongTensor] = None,
924         past_key_values: Optional[Union[Cache, List[torch.FloatTensor]]] =
None,
925         inputs_embeds: Optional[torch.FloatTensor] = None,
926         labels: Optional[torch.LongTensor] = None,
927         use_cache: Optional[bool] = None,
928         output_attentions: Optional[bool] = None,
929         output_hidden_states: Optional[bool] = None,
930         return_dict: Optional[bool] = None,
931     ) -> Union[Tuple, SequenceClassifierOutputWithPast]:
932         r"""
933         labels (`torch.LongTensor` of shape `(batch_size,)`, *optional*):
934             Labels for computing the sequence classification/regression
935             loss. Indices should be in `[0, ...,
936             config.num_labels - 1]`. If `config.num_labels == 1` a
937             regression loss is computed (Mean-Square loss), If
938             `config.num_labels > 1` a classification loss is computed
939             (Cross-Entropy).
940         """
941         return_dict = return_dict if return_dict is not None else
self.config.use_return_dict
942
943         transformer_outputs = self.model(
944             input_ids,
945             attention_mask=attention_mask,
946             position_ids=position_ids,
947             past_key_values=past_key_values,
948             inputs_embeds=inputs_embeds,
949             use_cache=use_cache,
950             output_attentions=output_attentions,
951             output_hidden_states=output_hidden_states,
952             return_dict=return_dict,
953         )
954         hidden_states = transformer_outputs[0]
955         logits = self.score(hidden_states)
956
957         if input_ids is not None:
958             batch_size = input_ids.shape[0]
959         else:
960             batch_size = inputs_embeds.shape[0]
961
962         if self.config.pad_token_id is None and batch_size != 1:
963             raise ValueError("Cannot handle batch sizes > 1 if no padding
token is defined.")
964         if self.config.pad_token_id is None:
965             last_non_pad_token = -1
966         elif input_ids is not None:
967             # To handle both left- and right- padding, we take the rightmost
968             token that is not equal to pad_token_id
969             non_pad_mask = (input_ids !=
self.config.pad_token_id).to(logits.device, torch.int32)
970             token_indices = torch.arange(input_ids.shape[-1],
device=logits.device)

```

```

967         last_non_pad_token = (token_indices * non_pad_mask).argmax(-1)
968     else:
969         last_non_pad_token = -1
970         logger.warning_once(
971             f"{self.__class__.__name__} will not detect padding tokens
in `inputs_embeds`. Results may be "
972             "unexpected if using padding tokens in conjunction with
`inputs_embeds`."
973         )
974
975     pooled_logits = logits[torch.arange(batch_size,
device=logits.device), last_non_pad_token]
976
977     loss = None
978     if labels is not None:
979         loss = self.loss_function(logits=logits, labels=labels,
pooled_logits=pooled_logits, config=self.config)
980
981     if not return_dict:
982         output = (pooled_logits,) + transformer_outputs[1:]
983         return ((loss,) + output) if loss is not None else output
984
985     return SequenceClassifierOutputWithPast(
986         loss=loss,
987         logits=pooled_logits,
988         past_key_values=transformer_outputs.past_key_values,
989         hidden_states=transformer_outputs.hidden_states,
990         attentions=transformer_outputs.attentions,
991     )
992
993
994 @add_start_docstrings(
995     """
996     The Llama Model transformer with a span classification head on top for
997     extractive question-answering tasks like
998     SQuAD (a linear layer on top of the hidden-states output to compute `span
999     start logits` and `span end logits`).
1000     """
1001     ,
1002     LLAMA_START_DOCSTRING,
1003 )
1004 class LlamaForQuestionAnswering(LlamaPreTrainedModel):
1005     base_model_prefix = "transformer"
1006
1007     # Copied from
1008     transformers.models.bloom.modeling_bloom.BloomForQuestionAnswering.__init__
1009     with Bloom->Llama
1010
1011     def __init__(self, config):
1012         super().__init__(config)
1013         self.transformer = LlamaModel(config)
1014         self.qa_outputs = nn.Linear(config.hidden_size, 2)
1015
1016         # Initialize weights and apply final processing
1017         self.post_init()
1018
1019     def get_input_embeddings(self):
1020         return self.transformer.embed_tokens
1021
1022     def set_input_embeddings(self, value):
1023         self.transformer.embed_tokens = value

```



```

1019 @add_start_docstrings_to_model_forward(LLAMA_INPUTS_DOCSTRING)
1020 def forward(
1021     self,
1022     input_ids: Optional[torch.LongTensor] = None,
1023     attention_mask: Optional[torch.FloatTensor] = None,
1024     position_ids: Optional[torch.LongTensor] = None,
1025     past_key_values: Optional[Union[Cache, List[torch.FloatTensor]]] =
None,
1026     inputs_embeds: Optional[torch.FloatTensor] = None,
1027     start_positions: Optional[torch.LongTensor] = None,
1028     end_positions: Optional[torch.LongTensor] = None,
1029     output_attentions: Optional[bool] = None,
1030     output_hidden_states: Optional[bool] = None,
1031     return_dict: Optional[bool] = None,
1032     **kwargs,
1033 ) -> Union[Tuple, QuestionAnsweringModelOutput]:
1034     r"""
1035         start_positions (`torch.LongTensor` of shape `(batch_size,)`,
*optional*):
1036             Labels for position (index) of the start of the labelled span
for computing the token classification loss.
1037             Positions are clamped to the length of the sequence
(`sequence_length`). Position outside of the sequence
1038             are not taken into account for computing the loss.
1039         end_positions (`torch.LongTensor` of shape `(batch_size,)`,
*optional*):
1040             Labels for position (index) of the end of the labelled span for
computing the token classification loss.
1041             Positions are clamped to the length of the sequence
(`sequence_length`). Position outside of the sequence
1042             are not taken into account for computing the loss.
1043         """
1044     return_dict = return_dict if return_dict is not None else
self.config.use_return_dict
1045
1046     outputs = self.transformer(
1047         input_ids,
1048         attention_mask=attention_mask,
1049         position_ids=position_ids,
1050         past_key_values=past_key_values,
1051         inputs_embeds=inputs_embeds,
1052         output_attentions=output_attentions,
1053         output_hidden_states=output_hidden_states,
1054         return_dict=return_dict,
1055     )
1056
1057     sequence_output = outputs[0]
1058
1059     logits = self.qa_outputs(sequence_output)
1060     start_logits, end_logits = logits.split(1, dim=-1)
1061     start_logits = start_logits.squeeze(-1).contiguous()
1062     end_logits = end_logits.squeeze(-1).contiguous()
1063
1064     loss = None
1065     if start_positions is not None and end_positions is not None:
1066         loss = self.loss_function(start_logits, end_logits,
start_positions, end_positions, **kwargs)
1067
1068     if not return_dict:
1069         output = (start_logits, end_logits) + outputs[2:]

```

```

1070         return ((loss,) + output) if loss is not None else output
1071
1072     return QuestionAnsweringModelOutput(
1073         loss=loss,
1074         start_logits=start_logits,
1075         end_logits=end_logits,
1076         hidden_states=outputs.hidden_states,
1077         attentions=outputs.attentions,
1078     )
1079
1080
1081 @add_start_docstrings(
1082     """
1083     The Llama Model transformer with a token classification head on top (a
1084     linear layer on top of the hidden-states
1085     output) e.g. for Named-Entity-Recognition (NER) tasks.
1086     """,
1087     LLAMA_START_DOCSTRING,
1088 )
1089 class LlamaForTokenClassification(LlamaPreTrainedModel):
1090     def __init__(self, config):
1091         super().__init__(config)
1092         self.num_labels = config.num_labels
1093         self.model = LlamaModel(config)
1094         if getattr(config, "classifier_dropout", None) is not None:
1095             classifier_dropout = config.classifier_dropout
1096         elif getattr(config, "hidden_dropout", None) is not None:
1097             classifier_dropout = config.hidden_dropout
1098         else:
1099             classifier_dropout = 0.1
1100         self.dropout = nn.Dropout(classifier_dropout)
1101         self.score = nn.Linear(config.hidden_size, config.num_labels)
1102
1103         # Initialize weights and apply final processing
1104         self.post_init()
1105
1106     def get_input_embeddings(self):
1107         return self.model.embed_tokens
1108
1109     def set_input_embeddings(self, value):
1110         self.model.embed_tokens = value
1111
1112 @add_start_docstrings_to_model_forward(LLAMA_INPUTS_DOCSTRING)
1113 @add_code_sample_docstrings(
1114     checkpoint=_CHECKPOINT_FOR_DOC,
1115     output_type=TokenClassifierOutput,
1116     config_class=_CONFIG_FOR_DOC,
1117 )
1118 def forward(
1119     self,
1120     input_ids: Optional[torch.LongTensor] = None,
1121     attention_mask: Optional[torch.Tensor] = None,
1122     position_ids: Optional[torch.LongTensor] = None,
1123     past_key_values: Optional[List[torch.FloatTensor]] = None,
1124     inputs_embeds: Optional[torch.FloatTensor] = None,
1125     labels: Optional[torch.LongTensor] = None,
1126     use_cache: Optional[bool] = None,
1127     output_attentions: Optional[bool] = None,
1128     output_hidden_states: Optional[bool] = None,
1129     return_dict: Optional[bool] = None,

```

```

1129     ) -> Union[Tuple, TokenClassifierOutput]:
1130         r"""
1131         labels (`torch.LongTensor` of shape `(batch_size,)`, *optional*):
1132             Labels for computing the sequence classification/regression
1133             loss. Indices should be in `[0, ...,
1134             config.num_labels - 1]`. If `config.num_labels == 1` a
1135             regression loss is computed (Mean-Square loss), If
1136             `config.num_labels > 1` a classification loss is computed
1137             (Cross-Entropy).
1138         """
1139         return_dict = return_dict if return_dict is not None else
1140         self.config.use_return_dict
1141
1142         outputs = self.model(
1143             input_ids,
1144             attention_mask=attention_mask,
1145             position_ids=position_ids,
1146             past_key_values=past_key_values,
1147             inputs_embeds=inputs_embeds,
1148             use_cache=use_cache,
1149             output_attentions=output_attentions,
1150             output_hidden_states=output_hidden_states,
1151             return_dict=return_dict,
1152         )
1153         sequence_output = outputs[0]
1154         sequence_output = self.dropout(sequence_output)
1155         logits = self.score(sequence_output)
1156
1157         loss = None
1158         if labels is not None:
1159             loss = self.loss_function(logits, labels, self.config)
1160
1161         if not return_dict:
1162             output = (logits,) + outputs[2:]
1163             return ((loss,) + output) if loss is not None else output
1164
1165         return TokenClassifierOutput(
1166             loss=loss,
1167             logits=logits,
1168             hidden_states=outputs.hidden_states,
1169             attentions=outputs.attentions,
1170         )
1171
1172 __all__ = [
1173     "LlamaForCausalLM",
1174     "LlamaModel",
1175     "LlamaPreTrainedModel",
1176     "LlamaForSequenceClassification",
1177     "LlamaForQuestionAnswering",
1178     "LlamaForTokenClassification",
1179 ]

```